

```
// Exemplo - Sem DRY (Errado)
0 referências
public class Errado_InvoiceService
{
    0 referências
    public void GenerateInvoice(string customerName, decimal amount)
    {
        Console.WriteLine($"Generating invoice for {customerName} with amount {amount}");
        Console.WriteLine("Saving invoice to the database...");
        Console.WriteLine("Sending invoice to the customer via email...");
    }

    0 referências
    public void GenerateReceipt(string customerName, decimal amount)
    {
        Console.WriteLine($"Generating receipt for {customerName} with amount {amount}");
        Console.WriteLine("Saving receipt to the database...");
        Console.WriteLine("Sending receipt to the customer via email...");
    }
}
```

Problemas:

- A lógica de "Salvar no banco de dados" e "Enviar email" está duplicada.
- Se houver uma mudança, será necessário alterar em múltiplos lugares.

```
//Exemplo - Com DRY(Certo)
2 referências
public class DocumentService
{
    2 referências
    public void SaveToDatabase(string documentType, string customerName, decimal amount)
    {
        Console.WriteLine($"Saving {documentType} for {customerName} with amount {amount} to the database...");
    }

    2 referências
    public void SendEmail(string documentType, string customerName)
    {
        Console.WriteLine($"Sending {documentType} to {customerName} via email...");
    }
}
```

```
1 referência
public class InvoiceService
{
    private readonly DocumentService _documentService;

    0 referências
    public InvoiceService()
    {
        _documentService = new DocumentService();
    }

    0 referências
    public void GenerateInvoice(string customerName, decimal amount)
    {
        Console.WriteLine($"Generating invoice for {customerName} with amount {amount}");
        _documentService.SaveToDatabase("Invoice", customerName, amount);
        _documentService.SendEmail("Invoice", customerName);
    }

    0 referências
    public void GenerateReceipt(string customerName, decimal amount)
    {
        Console.WriteLine($"Generating receipt for {customerName} with amount {amount}");
        _documentService.SaveToDatabase("Receipt", customerName, amount);
        _documentService.SendEmail("Receipt", customerName);
    }
}
```

Vantagens do Exemplo Certo:

1. **Reutilização de Código:** As ações repetitivas (salvar no banco e enviar email) foram movidas para uma classe reutilizável.
2. **Facilidade de Alteração:** Se o processo de envio de email mudar, será necessário ajustar apenas na classe `DocumentService`.
3. **Organização:** Cada classe tem uma responsabilidade clara.

Resumo

Evite escrever o mesmo código em múltiplos lugares. Sempre que identificar duplicação, pense em como encapsular a lógica em métodos, classes ou funções reutilizáveis. □